



HashLite – FPGA Cryptography

Project Engineering

Year 4

Gavin Walsh

Bachelor of Engineering (Honours) in Software and
Electronic Engineering

Atlantic Technological University

2024/2025

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Gavin Walsh – 26/04/2025

Acknowledgements

I would like to thank Ben Kinsella for the guidance and words of wisdom throughout the duration of the project.

Also, to all the ATU Software and Electronic Engineering faculty to get me where I am today.

Table of Contents

1	Summary	6
2	Poster	7
3	Introduction	8
4	The Inspiration	9
5	SHA-256 Overview	10
5.1	Some Areas its used:	10
5.2	How it works:	10
6	Tools and Technology	12
6.1	Verilog	12
6.2	Python	12
6.3	Vivado	12
6.4	Vitis.....	12
6.5	Digilent Basys 3	12
7	Project Architecture.....	13
8	Code	15
8.1	Padding	15
8.2	UART.....	16
8.2.1	Python Send	16
8.2.2	Verilog Receive.....	17
8.2.3	Verilog Send	18
8.3	SHA Functions	19
9	Results.....	20
10	Project Plan	22

11	Ethics	23
12	Conclusion	24
13	References	25
14	Learning Resources	Error! Bookmark not defined.

1 Summary

The goal of my project was to create a hardware-accelerated SHA-256 [3] [9] hashing system on an FPGA (Field Programmable Gate Array). The motivation behind the project was my interest in FPGA development and learning more about the technology.

The scope of the project was to make a full synthesized SHA-256 core in Verilog. This core would communicate with a host PC via UART for data transfer. This Core was deployed on a Digilent Basys 3 [3] board, using a Xilinx Artix-7 FPGA.

Key features are the SHA-256 core that handles the hashing of the messages. The bespoke UART interface for getting messages on and off the FPGA as efficiently as possible, and finally the Python script that prepares the data to be sent and received.

Throughout the project I used many different technologies. As for software, Verilog was used for the SHA core and the UART interface, Python was used for communication scripts. C was used in an early version to program a microprocessor in place of the custom UART module. Hardware wise the Digilent Basys 3 [3] was used. For IDE's all my python was developed in PyCharm, the Verilog in Vivado which also enabled me to use the block design for dragging in softcores, clocks, An Integrated Logic Analysers (ILA) for live signal verification and debug, and Vitis to program the microprocessor when it was in use.

What was accomplished was a successful implementation of a SHA-256 core on the FPGA, capable of efficiently hashing input data from a host PC while ensuring minimal latency. The project demonstrated the advantages of hardware acceleration for specific workloads.

2 Poster



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

HashLite FPGA Cryptography

Gavin Walsh – G00402296

BEng (H) in Software & Electronic Engineering

Introduction

What is SHA?

SHA-256 (Secure Hash Algorithm) is a cryptographic hash function that generates unique 256-bit outputs, widely used in security applications like blockchain, digital signatures, and data verification.

How SHA2 Works:

- Message Preprocessing** – The input message is padded to ensure it fits into fixed 512-bit blocks.
- Message Parsing** – The padded message is divided into 512-bit blocks for processing.
- Initialization** – The algorithm starts with predefined hash values and constants.
- Compression Function** – Each block undergoes multiple rounds of bitwise operations, including shifts, XORs, and modular additions, using a set of constants and functions.
- Final Hash Output** – After all blocks are processed, the final 256-bit hash value is produced, serving as a unique digital fingerprint of the original message.



What are FPGAs?

FPGA (Field-Programmable Gate Array) is a customizable hardware chip that can be programmed to perform specific tasks, offering the flexibility of software with the speed and efficiency of dedicated hardware.

Project Summary

Summary:

This project implements SHA-256 hashing on an FPGA using a MicroBlaze soft-core processor. The Host PC sends data via UART, where the MicroBlaze processor manages communication between the SHA256 module and on-board memory. The benefits of this approach include improved speed and efficiency, as FPGA parallelism accelerates hashing compared to software-based implementations.

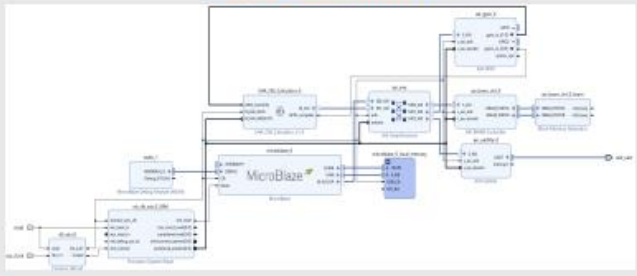
How It Works:

- User Input:** The host PC provides data through a user interface. That gets sent to the FPGA via UART
- MicroBlaze Microprocessor:** Microprocessor loads the message into a shared memory area for the module to process.
- SHA256 Computation:** The SHA256 module processes the data and generates the hash. Leaving the data in a shared memory area.
- Output Generation:** The final hash result is sent back to the host PC.

Performance Metrics:

- Achieved a clock frequency of 50 MHz
- Utilized 2,540 Look-Up Tables (LUTs), representing 12% of available resources
- Utilized 2,416 Registers, accounting for 6% of available resources
- Power Consumption: The design operates at a power usage of 0.206 Watts

Project Architecture



Key Components

- SHA-256 Calculator:** Module for performing SHA-256 hashing computations, generating hash outputs.
- AXI (Advanced eXtensible Interface) SmartConnect:** Hub that routes data between the MicroBlaze, SHA-256 module, and memory.
- AXI UARTlite:** Provides a UART interface for serial communication with the host PC.
- AXI BRAM Controller:** Manages access to Block RAM for temporary data storage during processing.

Technologies Used

Software Languages:

- Verilog** - Hardware description language for SHA-256 implementation
- C** - Programming for FPGA microprocessor control
- Python** - PC-to-FPGA serial communication interface

Hardware:

- Digilent Basys 3** - FPGA development board featuring Xilinx Artix-7 FPGA

Development Tools:

- Vivado** - FPGA design and synthesis suite
- Vitis** - Embedded software development platform

Design Layout

This design layout showcases the arrangement of electronic components.

Blue Areas: These represent the logic blocks, including They are responsible for executing computations and processing data.

Yellow Areas: These sections are for routing resources that facilitate communication between different components.

Green Areas: These represent memory blocks, such as Block RAM



3 Introduction

The ever-increasing demand for faster and more energy efficient computing has sparked my interest in FPGAs and hardware solutions for software problems. I pursued this project to investigate the practical applications of hardware acceleration, specifically how custom hardware implementations can enhance the efficiency of complex computations like AI, Cryptography, Simulation. These areas are always changing making FPGAs a strong candidate to fix these problems. This project provided some hands-on learning in a field I am passionate about.

The goal was to create a hardware implementation SHA-256 hash function, showcasing the performance gains over traditional software. Through this project, I gained practical experience in optimizing hardware designs for performance and resource utilization, including techniques like pipelining, resource sharing and the issues like the disconnect between simulation and real life, timing issues, signal degradation and the longer development and test time compared to software.

I chose to implement a hardware-accelerated SHA-256 hash function to demonstrate the upsides of hardware compared to software. SHA-256's computational intensity and the algorithms simplicity to understand and implement in HDL on paper. The project involved designing the SHA-256 core in Verilog including the message scheduling, message compression and the output, building a communication interface so the FPGA could talk to the computer this took many iterations starting life as MicroBlaze reading and writing to a shared memory area of block ram through an AXI (Advanced eXtensible Interface) [1][2] then later I moved to directly communicating over UART with a HDL module removing some of the bloat, and writing a Python script to handle the data and time for verification purposes.

4 The Inspiration

As I have said that I have a big interest in FPGA design since I was introduced to it in my System on chip module as at apart of my studies. This only gave me a glimpse into what is possible, soon enough I was getting videos online recommended to me of people using FPGAs to emulate older gaming consoles. Delving deeper into this topic there is a large community of developers who reverse engineer older gaming console and give them new leases of life by improving the design like adding more memory, adding HDMI output for modern TVs and some even do upscaling making it possible to play the originally low-resolution games in 1080p although it's not a real 1080p. Why this is great, some consoles are growing hard to get the older they become, and things break over time. It can be difficult to get chips from even 20 years ago that were bespoke for this console. Why hardware emulation is typically better than software, some games that were made for only one console will base all their timings off the frames per second so unless you have a high-end pc you will see lots of issues and possibly soft lock yourself in the game. This community has sold me on the power of FPGAs as all you need to do is get a specific dev board and you have so many different consoles.

This is where the idea of hardware accelerators came from to learn how people make these. I thought making an emulator would be a bit above my paygrade, so I opted for the SHA256 something well documented and tangible.

5 SHA-256 Overview

SHA-256 [9] [3] (Secure Hash Algorithm 256-bit) is a very popular hashing function that is used mostly for data verification and security. It takes an input and returns a unique 256-bit output. A key characteristic is that it's very collision resistant meaning its very unlikely for 2 inputs to give the same output.

5.1 Some Areas its used:

- **Data Integrity:** By comparing the hash of the data before and after send or storage, you can detect any changes.
- **Digital Signatures:** SHA-256 is used to create a digital signature, which be used to verify authenticity of digital documents.
- **Password Storage:** Systems often store their SHA-256 hashes, making it harder to retrieve the original passwords even if the database is hacked.
- **Blockchain Technology:** SHA-256 is a fundamental part to blockchain technologies, where it's used to link and secure blocks in the chain.

5.2 How it works:

1. **Data Padding:** The data is split into 512-Bit blocks if the message is less than 512 it gets 0 padded. Then the length of the original message is appended to the end.
2. **Message Expansion:** The message gets divided into 16 32-Bit words. They are expanded into 64 32-Bit words through a series of bitwise XOR, Rotations and Bit Shifts.
3. **Message Compression:** Each of the 64 rounds involves a series of calculations using the 64 words, round constants (cube roots of the first 64 prime numbers) and bitwise operations like the expansion rounds.
4. **Output:** The final hash value is a 256-bit hexadecimal string from the compressions.

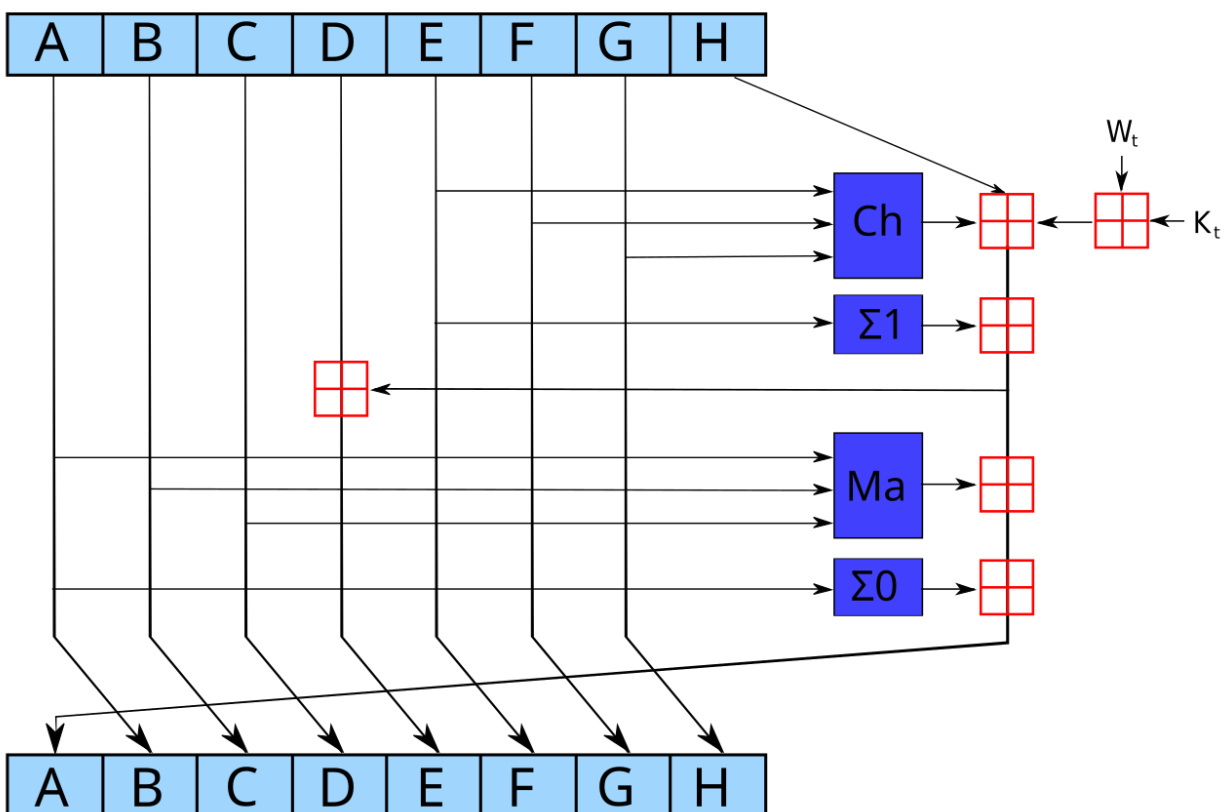


Figure 5.1 – Compression Function [9]

6 Tools and Technology

6.1 Verilog

Verilog is a Hardware Description Language, a used language for designing and describing the SHA-256 core and the UART module. This was choosing because of the familiarity from my System on Chip module.

6.2 Python

Python is a high-level programming language. It was chosen for this for its easy as it's easy to read, plenty of documentation and my previous experience. This made it simple to make changes as my project progressed.

6.3 Vivado

Xilinx Vivado, the design suite used for synthesizing the Verilog code, implementing the design on the Basys 3 [3] FPGA and for performing timing analysis and debugging.

6.4 Vitis

Was used to develop the software that interacted UART and block ram when they were in use.

6.5 Digilent Basys 3

This was all deployed on a Digilent Basys 3 [3], this was chosen because of my familiarity with the board, it also provided plenty of resources for this implementation like memory, flipflop and a high enough clock.

7 Project Architecture

Figure 7.1 shows the high-level system architecture. The Python script running on the host PC sends the input message to the FPGA via the UART. The UART module passes the message to the SHA-256 core, processes the message. The hash digest is returned to the PC, where the Python script verifies the hash against a known good value and the performance is logged.

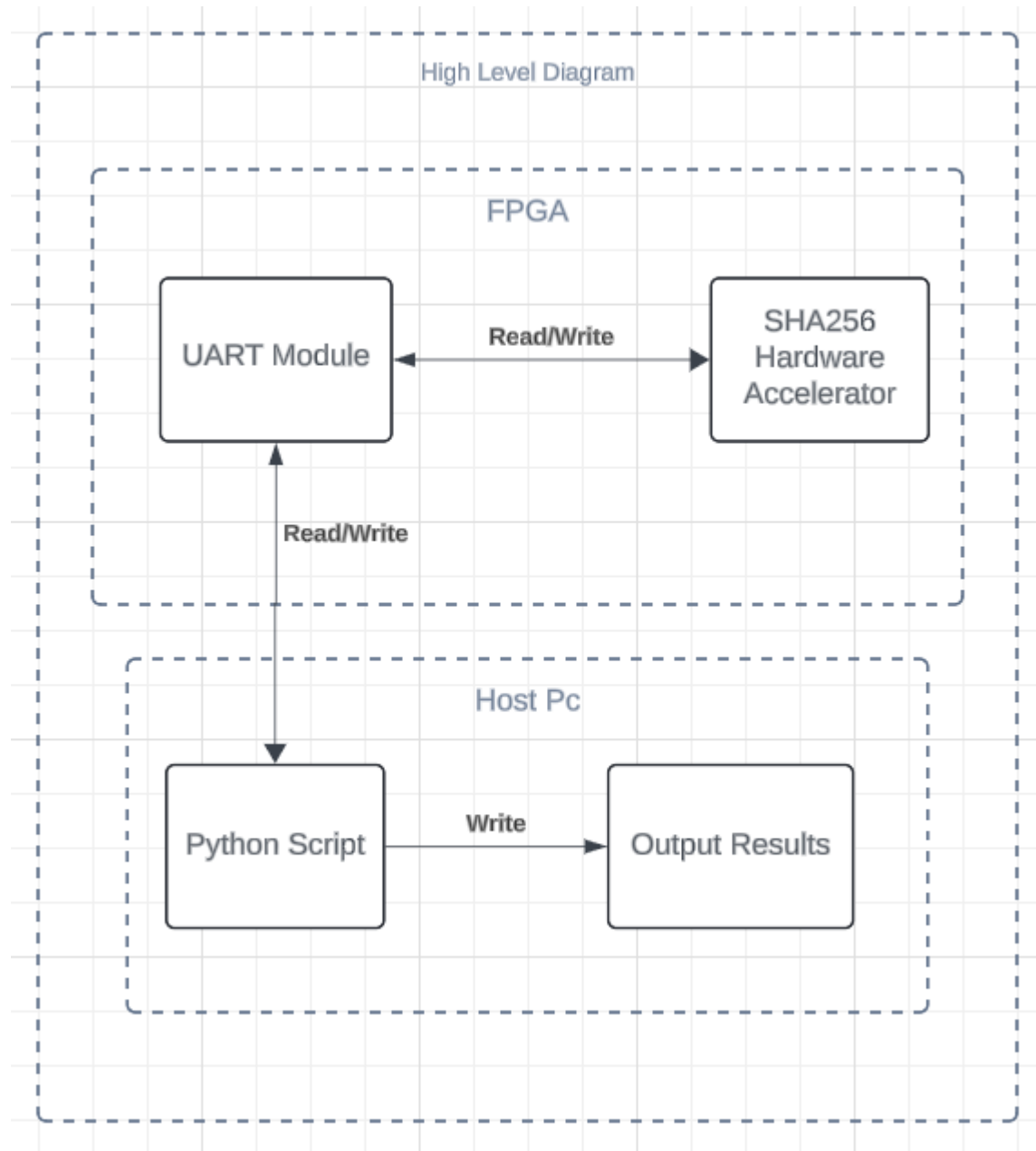


Figure 7.1 High level Architecture Diagram

Figure 6.2 gives a detailed view of the FPGA's structure. The SHA-256 core receives the input message data via the `message_input` bus and outputs the calculated hash on the `hash_output`. Control signals `hash_start` and `hash_done` manage the hashing process. The `hash_state` will reset the working variables if needed. The UART module handles serial communication with the host PC, transmitting and receiving data via the `tx` and `rx` signals.

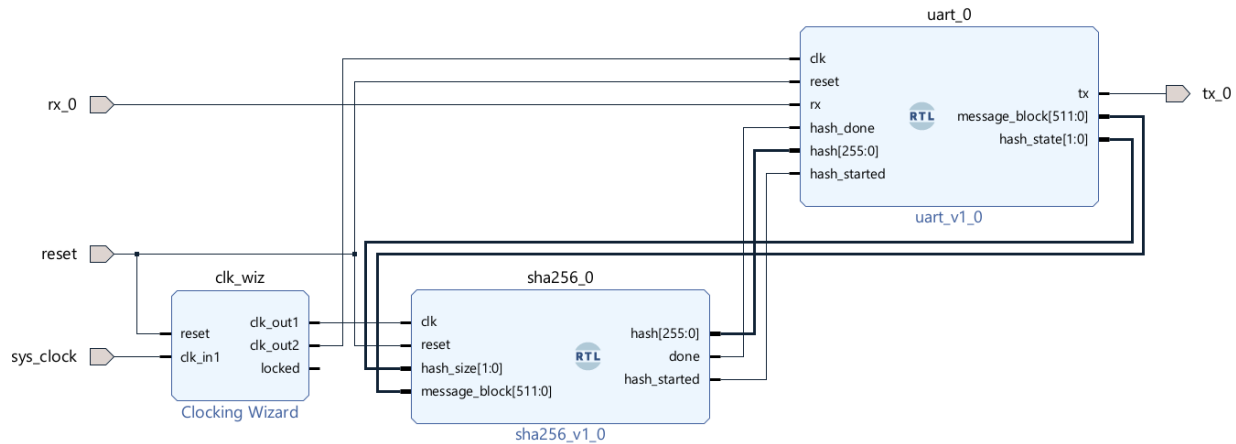


Figure 7.2 Low level FPGA Block Diagram

8 Code

This section will show the important design decisions I made, along with the drawbacks of the way I did things.

8.1 Padding

This is the part where the data gets padded out to the messages and we append the length. I decided to do this on the host PC, so we weren't limited to the 36K-bits of onboard memory on the FPGA. I use `struct.pack` to make sure that the bytes are always a big-endian unsigned long making sure the most significant bit was saved first.

```
def preprocess(message_input): 1 usage 2 Gavin *  
    """  
    Padding Stage  
    """  
    message = bytearray(message_input, 'utf-8')  
    original_length_bits = len(message) * 8  
    message.append(0x80)  
    while len(message) % 64 != 56:  
        message.append(0)  
    message += struct.pack( fmt: '>Q', *v: original_length_bits)  
    chunk_size = 64  
    chunks = [message[i:i + chunk_size] for i in range(0, len(message), chunk_size)]  
  
    return chunks
```

Figure 7.1.1 – Frontend Structure

8.2 UART

8.2.1 Python Send

The next stage is to send the message blocks to the FPGA. We use the previous preprocess function then append if a 'S' if we want an immediate reply or an 's' if there is more to come. Next, we send the block in reverse as I didn't want to change the preprocess function. When if I would be switching back and forth between the old and new implementations. If it is a large hash, we append an 'L' at the end signalling we are expecting a reply.

```
def send_block(text, uart): 1usage new *
    try:
        blocks = preprocess(text)
        print("Chunks", len(blocks))
        if len(blocks) == 1:
            block_ending = 'S'.encode()
        else:
            block_ending = 's'.encode()
        for j, block in enumerate(blocks, start=1):
            uart.write(block[::-1] + block_ending)
            uart.flush()
            if j < len(blocks):
                wait_for_ready(uart)
            if j == len(blocks) - 1:
                block_ending = 'L'.encode()
            else:
                block_ending = 'l'.encode()
```

Figure 8.2.1

8.2.2 Verilog Receive

This Verilog Code shows the UART [2] receive once the start bit is got, we move on to storing all the bits into an array. After that we assign the values to the `message_block` and start the hashing. Then after we will parse the command character sent with the message. If it was a capital the system will respond when the hash is done.

```
START_BIT: begin
    if (clk_count == 0) begin
        if (rx == 1'b0) begin
            state <= DATA_BITS;
            clk_count <= CLKS_PER_BIT - 1;
            bit_count <= 4'b0;
        end else
            state <= IDLE;
        end else
            clk_count <= clk_count - 1;
    end

DATA_BITS: begin
    if (clk_count == 0) begin
        clk_count <= CLKS_PER_BIT - 1;
        data_byte[bit_count] <= rx;
        if (bit_count == 7)
            state <= STOP_BIT;
        else
            bit_count <= bit_count + 1;
        end else
            clk_count <= clk_count - 1;
    end

STOP_BIT: begin
    if (clk_count == 0) begin
        if (rx == 1'b1) begin
            if (byte_count < 64) begin
                message_block[byte_count*8 +: 8] <= data_byte;
                byte_count <= byte_count + 9'b1;
            end else if (byte_count == 64) begin // command
                case (data_byte)
                    8'h73: begin // 's'
                        hash_state <= SIGNLEHASH;
                    end
                end
            end
        end
    end
end
```

Figure 8.2.2

8.2.3 Verilog Send

Figure 8.2.3 shows the UART send functions on the FPGA we send 'FF' when the RX is ready for the next message, we send the hash when the done signal is high. The CLKS_PER_BIT variable is defined by the integer value of System Clock Frequency / BAUD rate.

```

case (tx_state)
  TX_IDLE: begin
    if (send_response && hash_done) begin
      tx_state    <= TX_SENDING_HASH;
      tx_active    <= 1;
      tx          <= 0; // Start bit
      tx_clk_count <= CLKS_PER_BIT - 1;
      tx_bit_count <= 0;
      tx_byte_count <= 0;
    end else if (!tx_active && state == IDLE) begin
      tx_active    <= 1;
      tx          <= 0; // Start bit
      tx_data      <= 8'hFF;
      tx_clk_count <= CLKS_PER_BIT - 1;
      tx_bit_count <= 0;
    end
  end
end

  TX_SENDING_HASH: begin
    if (!tx_active) begin
      if (tx_byte_count < 32) begin // 256 bits = 32 bytes
        tx_active    <= 1;
        tx          <= 0;
        tx_data      <= hash[tx_byte_count*8 +: 8];
        tx_clk_count <= CLKS_PER_BIT - 1;
        tx_bit_count <= 0;
        tx_byte_count <= tx_byte_count + 1;
      end else begin
        tx_state <= TX_IDLE;
      end
    end
  end
end
endcase

if (tx_active) begin
  if (tx_clk_count > 0) begin
    tx_clk_count <= tx_clk_count - 1;
  end else begin
    tx_clk_count <= CLKS_PER_BIT - 1;
    if (tx_bit_count < 8) begin
      tx          <= tx_data[tx_bit_count];
      tx_bit_count <= tx_bit_count + 1;
    end else if (tx_bit_count == 8) begin

```

Figure 8.2.3

8.3 SHA Functions

Symbols meaning:

& = bitwise and ^ = bitwise XOR {x [1:0], x [31:2]} = Right Rotate

W = array of working variables K = SHA constants

```
// SHA-256 functions
assign ch = (e & f) ^ (~e & g);
assign maj = (a & b) ^ (a & c) ^ (b & c);
assign s0 = {a[1:0],a[31:2]} ^ {a[12:0],a[31:13]} ^ {a[21:0],a[31:22]};
assign s1 = {e[5:0],e[31:6]} ^ {e[10:0],e[31:11]} ^ {e[24:0],e[31:25]};
```

Figure 8.3.1

```
// Message schedule functions
wire [31:0] sigma0, sigma1;
assign sigma0 = {W[round_count-15][6:0],W[round_count-15][31:7]} ^
                {W[round_count-15][17:0],W[round_count-15][31:18]} ^
                (W[round_count-15] >> 3);

assign sigma1 = {W[round_count-2][16:0],W[round_count-2][31:17]} ^
                {W[round_count-2][18:0],W[round_count-2][31:19]} ^
                (W[round_count-2] >> 10);
```

Figure 8.3.2

```
// T1 and T2 calculation
wire [31:0] t1_next = h + s1 + ch + K[round_count] + w_next;
wire [31:0] t2_next = s0 + maj;
```

Figure 8.3.3

9 Results

In figure 9.1 we see time vs the size in bytes of the FPGA implementation against the implementation I made in python on my PC. To get these data point I used the same vectors to keep it as fair as possible. This gave some surprising results with the two implementations being within 0.0003 seconds of each other and both having a slope of 0.000002. I did have to run the tests a few times as I thought I was doing something wrong but this rings true. I did try improving the FPGA's performance once again but sadly we are limited by the UART.

Computer Analysis:

Slope: 0.000002

Y Intercept: 0.000878

FPGA Analysis:

Slope: 0.000002

Y Intercept: 0.008729

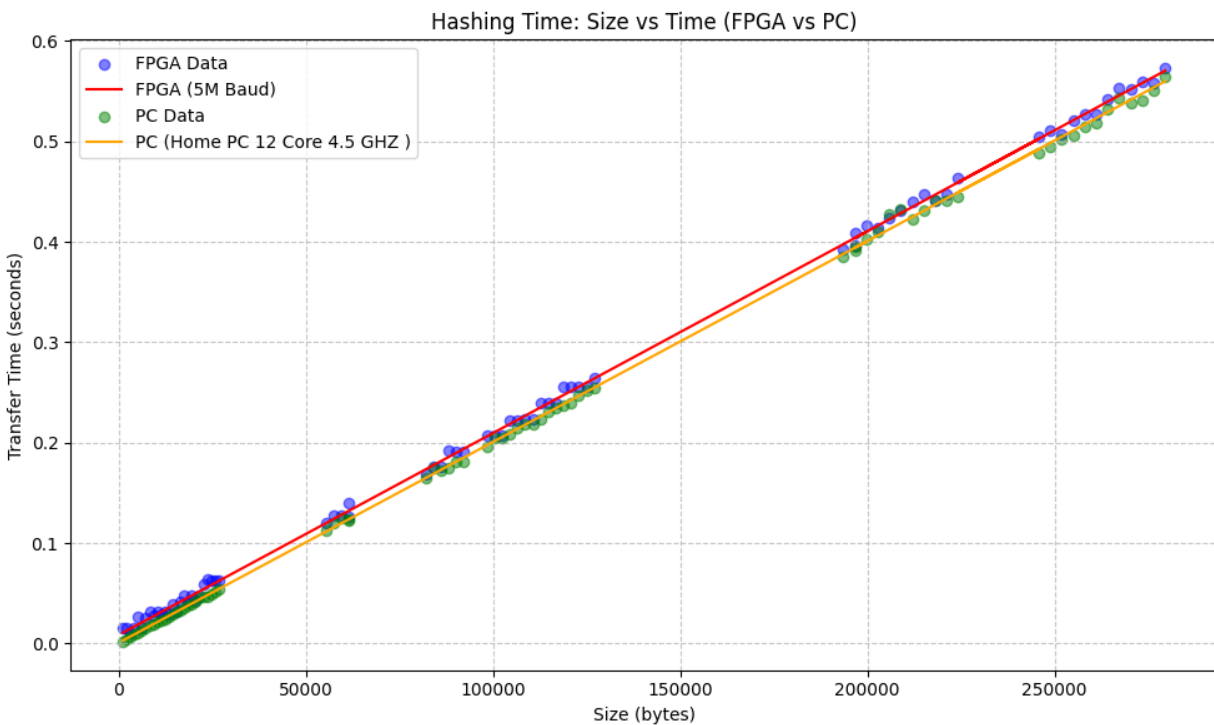


Fig 9.1 Results

The SHA256 implementation takes 65 clocks to complete and runs at 50mhz.

SHA Timing:

Period = 20ns Clock Cycles to complete = 65

Total time = $20 * 65 = 1300\text{ns}$

UART Timing:

Period for 1 bit = 200ns (5M Baud) Number of bits per transaction = $8 + 1 + 1 * (512/8) = 640$

Total time = $640 * 200 = 128,000\text{ns}$

This means the SHA is idle close to 99% of the time. I opted for UART originally using the Microblaze microcontroller as the UART controller, but I couldn't get the data speed I wanted the max being 115200 is all I could achieve this prompted the redesign which brought the hash time down exponentially.

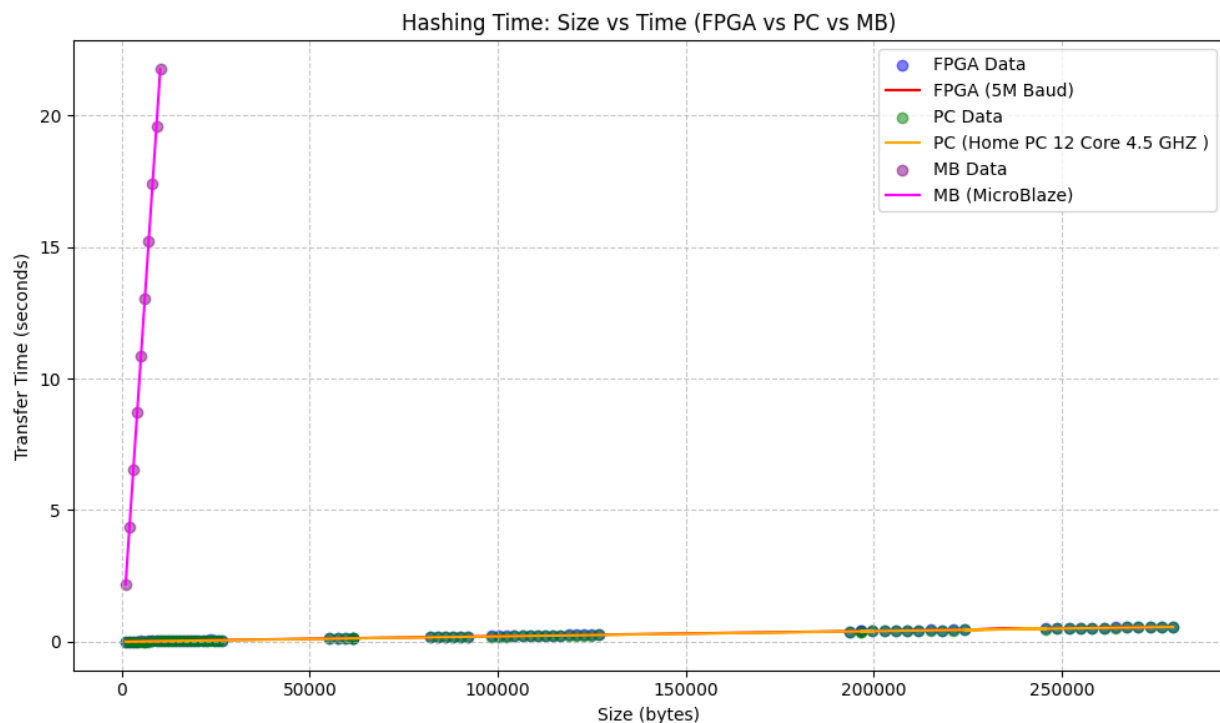


Fig 9.2 MicroBlaze Implementation Results

10 Project Plan

I used Jira as my project management tool. My scrum philosophy was Kanban, I chose this approach as the timeline for this project was hard to define. I created a backlog of steps then gave an amount of time that I thought was appropriate and did the task updating as I completed the task.

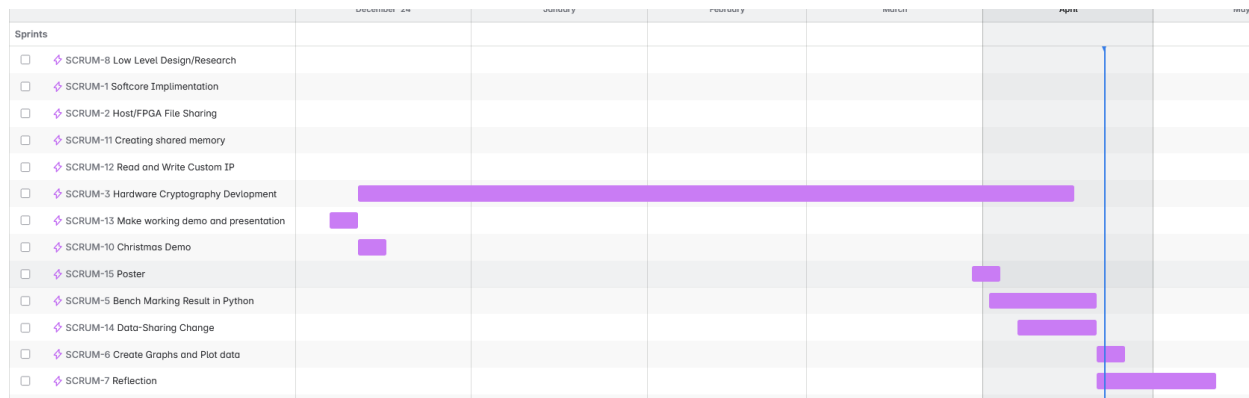


Fig 10.1 Jira Roadmap

11 Ethics

Hashlite mainly focusses on improving the speed of a cryptography algorithm to be used to secure data, but it could also be used in reverse. If someone was to get their hand on a database with passwords stored as SHA hashes one could brute force guess the passwords until the hash comes out the same. To do a better job we could run a second hash on the password making it very difficult to work backwards to the original data.

Aside from the cryptography the goal was to make a computationally heavy task both faster and hopefully take less electricity in doing so. For this specific application I don't see many benefits as there are more efficient ASIC designs but if this were to do a different more bespoke task there could be massive power usage reductions.

12 Conclusion

This project was probably one of the best ways for me to get familiar with this technology. While my implementation is not going to revolutionize the world. It provided me with a deep understanding of FPGA design flows. I also got very familiar with the complexities of hardware debugging this platform and realised that a solid plan is very important and to build things in manageable chunks rather than trying to do everything and have nothing work. I genuinely found this to be a challenge and spent more time reading and learning than coding.

As for the results I am happy with how it turned out Knowing that I am limited more by the hardware is helpful. Possibly moving to a high-speed data transfer protocol like ethernet or PCIe.

13 References

- [1] ARM, AXI Protocol Specification. [Online]. Available: <https://developer.arm.com/documentation/ih0022/e/> Accessed: Oct 10, 2024.
- [2] AMD, AXI GPIO Specification. [Online]. Available: <https://docs.amd.com/v/u/en-US/pg144-axi-gpio> Accessed: Oct 10, 2024
- [3] National Institute of Standards and Technology, Secure Hash Standard. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf> Accessed: Oct 18, 2024.
- [4] Digilent, Basys 3 Reference Manual, Digilent Reference. [Online]. Available: <https://reference.digilentinc.com/basys3/refmanual> . Accessed: Oct 2, 2024.
- [5] ChatGPT. [Online]. Available: <https://openai.com/chatgpt>. Accessed: Oct 1, 2024.
- [6] Digital Hardware Design, SHA256 in Hardware - FPGA Verilog Implementation. [Online]. Available: <https://www.youtube.com/watch?v=UqXXWI28HyQ>. Accessed: Oct 16, 2024.
- [7] Project FPGA, SHA256. [Online]. Available: <https://projectfpga.com/sha256/code.php> Accessed: Oct 12, 2024.
- [8] Wikipedia, SHA-2. [Online]. Available: <https://en.wikipedia.org/wiki/SHA-2>. Accessed: Oct 18, 2024